ARTICLE

Dynamic Dispatching Rule Selection for the Job Shop Problem

Nuno Marques[a,b], Gonçalo Figueira[a,b], Luis Guimarães[a,b]

[a]INESC TEC [b]Faculdade de Engenharia, Universidade do Porto

ARTICLE HISTORY
Compiled July 19, 2024

ABSTRACT
Uncertainty is permanent in modern manufacturing settings. To cope with it, scheduling decisions are commonly taken resorting to dispatching rules, which are reactive in nature. However, rule performance varies according to shop load and job urgency. Therefore, the scope of this work is to select the most appropriate dispatching rule as conditions change in dynamic job shops. Such is achieved by proposing periodic and real-time dispatching rule selection systems. The former selects the rule after every period and is trained via supervised learning. The latter selects the most suitable rule every time a scheduling decision is necessary and is trained via reinforcement learning. The models outperformed benchmark dispatching rules by over 20%. All in all, results show that there is a proper incentive for managers to adopt dispatching rule selection systems.

KEYWORDS
Dynamic Scheduling; Job Shop; Dispatching rules

1. Introduction

Managing operations in modern manufacturing settings has become an increasingly complex task due to more complicated products, shorter life cycles and frequent occurrence of uncertain events (Yin et al. 2017). To face these challenges, managers are pushing for Industry 4.0 (I4.0), which is expected to grow at a 15% CAGR until 2030 (Subramanian 2024). Under this paradigm, production settings are becoming Cyber-Physical Systems (CPSs) where physical and virtual worlds interact seamlessly with the support of state-of-the-art technologies (Xie et al. 2019). CPSs can monitor the facilities in real time with little human intervention, reacting to uncertain events within milliseconds (Khajeh et al. 2022).

Scheduling is a critical task in manufacturing. However, it must be aligned with the new manufacturing reality in which there is little time to react. Therefore, in an effort to adjust to increasing levels of uncertainty, Dispatching Rules (DR) have been preferred to exact and metaheuristic methods due to their reactive nature (Golmakani and Birjandi 2013). Additionally, it has been shown that exact methods do not bring quality improvements over DRs in highly dynamic and uncertain environments (Rajabinasab and Mansour 2011).

Over the years, researchers designed several DRs for different conditions and optimisation criteria. However, in spite of their best efforts, no DR dominates all others

CONTACT Nuno Marques. Email: up201503975@edu.fe.up.pt

(Baker 1984). Instead, DR Selection Systems (DRSSs) that dynamically adjust the DR in use as conditions change emerged in the literature (Priore et al. 2018).

Two main types of DRSS have been proposed. One type, based on Reinforcement Learning (RL), selects the rule in real-time (RTRS). This recent RL-based approach challenges the well-established approach that selects the DR at fixed periods, PRS, and is based on Supervised Learning (SL). Up until this moment, there is scant literature comparing these two approaches.

Furthermore, it is unclear from the literature how to define the features that describe the state of the shop floor and the DRs that DRSS rely on. Most works present state sets without a great explanation concerning feature selection. Regarding the action set, previous works rely on simple DRs (such as the ones present in Table 12) or custom DRs (e.g.: Ahn and Hur 2020; Wei and Zhao 2004). It is not clear that the elected DRs are true alternatives between themselves.

Motivated by the gaps highlighted above, this work aims to improve DR-based scheduling by proposing DRSSs for dynamic Job Shops (JSs). All the main components of a DRSS are subject to thorough exploration. To start, a compact state set is found by analysing the correlation between features. Moreover, a complementary action set is established by simulating DRs in diverse conditions.

Regarding system agents, models for both PRS and RTRS are proposed, based on SL and RL, respectively. Agent performance is compared in a wide range of JS instances, including stress situations. Finally, a fifth component is added to the system, the flusher, that is responsible for identifying operations that are stuck.

In the following section, Section 2, the dynamic JS problem is presented in detail. Then, in Section 3, the competing DRSS approaches that emerged in the literature are introduced. Section 4 is dedicated to the methodology, where the main components of a DRSS are presented. In section 5, the comprehensive set of instances that were used in computational tests are described, as well as the parameters and hyper-parameters of the proposed models. Section 6 starts by showing how the action and state sets were defined. Afterwards, the performance of the proposed DRSSs are assessed. It includes a comparison against existing DRs and the impact on the performance of different parameters. The paper ends with Section 7, where the main outcomes of the paper are highlighted.

2. The dynamic job shop scheduling problem

In the dynamic job shop scheduling problem, a set of jobs J arrives randomly at the JS to be processed in a pool of machines M. Each job $j \in J$ has a set of operations O_j that must be processed in a given sequence. Each operation $o \in O_j$ takes p_{oj} units of time to be completed, and must be processed in a particular machine without interruption. Machines process one operation at a time. The goal is to minimise a given criterion such as makespan, time taken to complete all jobs in the JS, or flowtime, time jobs stay at the JS. In this work, tardiness, which is the time jobs take beyond their due date, is the metric to optimise.

The role of DRs in the JS problem is to sort the queue of machines when these finish processing an operation. DRs compute a score for each operation in the queue of the machine, according to the DR expression. The operation with the most advantageous score is chosen to be processed next.

Over the years, several DRs have been proposed for various optimisation objectives and JS conditions. However, no rule clearly dominates others (Park et al. 1997).

For the particular case of *Tardiness* optimisation, researchers faced a great challenge as DR performance crossovers for different utilisation and due date tightness levels (Baker 1984). With due-date features leading to better performance in low utilisation or loose due-dates and processing times related features suitable for high intensive load (Ferreira et al. 2022). Therefore, some DRs that have been proposed aim to balance the relative importance of these two dimensions. For example, Raghu and Rajendran (1993) propose the RR-rule, described by Equation 1, that relies on the utilisation factor, u, as a weight for the two components. In spite of rendering good results in general, the RR-rule is still outperformed by the rule proposed by Holthaus and Rajendran (2000), $2PT + NPT + WINQ$, in tight load conditions. As different DRs are suitable for specific shop operating conditions, researchers have proposed systems that dynamically select the active DR as conditions change, the DRSS mentioned above.

$$PT \cdot e^{u} + PT \cdot \frac{Slack}{RPT} \cdot e^{-u} + WINQ \tag{1}$$

3. Previous works

Initial approaches, such as Wu and Wysk (1989), selected the DR by simulating the performance of several candidate rules. The simulation step is time-consuming and thus violates the reactive behaviour of DRs. More recently, Durasević et al. (2023) explored DR selection by resorting to simulation. Even with more powerful modern hardware, simulation was much slower than a DR (3x to 35x slower).

Researchers turned to machine learning to help with the dynamic selection of DRs. Two main approaches have risen, PRS and RTRS. The former tackled with SL and the latter with RL.

Periodic rule selection

Over the years, several SL algorithms were proposed to tackle the PRS problem such as Decision Trees (e.g. Park et al. 1997), Neural Networks (e.g. Min and Yih 2003), and Support Vector Machines (e.g. Liu et al. 2005). These works span various job shop settings, for example, there are DT-based PRS systems for the job shop (e.g. Metan et al. 2010), flexible job shop (e.g. Park et al. 1997), and flexible flow shop problems (e.g. Choi et al. 2011).

Metan et al. (2010) proposes a DT-based PRS agent for the dynamic job shop problem. Overall, 26 system features are presented, which are selected by a backward step selection heuristic. An example DT is presented, and it relies on thirteen features. An action set with three DRs is used (SPT, MDD, ODD) and a set with a fourth DR is also tested (MOD). Six different DR selection frequencies were assessed. Additionally, when the system detects that the DT performance is deteriorating significantly, a new DT would is trained. The proposed system was found to outperform multipass simulation technique and several established DRs in the vast majority of the scenarios.

Park et al. (1997) tackles the flexible job shop problem using two DTs. One DT is responsible for selecting the DR over time. The other DT is concerns the period a dispatching rule is active, as this parameter also depends on the system conditions, a kind of variable period approach. The agent relies on three states that concern system utilisation, allowance and flexibility and four DRs (EDD, SPT, MOD, MDD). The system is effective in reducing mean tardiness, particularly in instances in which machine breakdowns and urgent jobs are present.

Choi et al. (2011) proposes a DT-based PRS for a TFT-LCD plant modelled as an hybrid flow shop. The system state is described relying on four variables, namely, remaining number of operations and remaining processing times for the jobs being processed and the jobs in queue. The PRS agent can choose among fourteen elementary DRs. DR selection occurs at very specific moments, the beginning of a scheduling period, when a machine breakdown, and when a period ends, but only if the performance worsens beyond a given threshold. Four different period lengths were tested. The agent displayed competitive performance in several criteria, namely, system throughput, mean flow time, mean tardiness, and the number of tardy jobs.

Min and Yih (2003) explores a NN-based PRS model for a semiconductor production plant in Korea. The system relies in five state features which concern the remaining number of operations, the remaining processing time, the load in critical machines, the work in progress, and slack times. Seven DRs are available FIFO, EDD, RPT, CR, the remaining are related with buffer sizes. DRs are selected for each of the plant bays at the beginning of each shift. The agent outperformed both the random policy and the FIFO rule in the mean flowtime, mean of slack time, and mean remaining processing times.

Liu et al. (2005) proposes a SVM-based PRS agent for a flexible job shop. The state is comprised by 26 features and the agent has 7 standard DR to choose from (FIFO, SPT, RPT, CR, SLACK, EDD, SIO). The DR is selected for each production day. A 4% improvement throughput-wise is reported when compared to the best DR (SPT).

In a later stage, as the DRSS literature matured, authors started comparing the performance of the several existing SL algorithms Shiue et al. (e.g. 2012); Priore et al. (e.g. 2018); Durasević and Jakobović (e.g. 2022). In some cases, ensembles of SL-models were proposed.

Shiue et al. (2012) explored several SL algorithms regarding the task of finding the most suitable rule at each scheduling period in a flexible job shop. The work includes Decision Trees (DTs), NNs, SVMs, finally, Ensembles of the previous models. The training dataset was built based on multipass training. The work considers a large number of features to describe the state space (30 features), so a GA algorithm is used to find the most desirable features. The models can employ a set of well-known simple DRs, namely, SPT, EDD, SLACK, FIFO, SIO. Computational tests showed that the models were more effective than individual DRs in the optimisation criteria of throughput, mean cycle time and number of tardy parts.

In Priore et al. (2018), several supervised learning algorithms were proposed for PRS in a flexible job shop. Apart from the already explored DT, SVM, NN and Model Ensemble, this work also included k-NN. The models could use eight DRs, four routing rules (SPT, NIQ, WIQ, LUS - lowest utilised station) and four sequencing rules (SPT, RPT, EDD, MDD). All routing-sequencing rules possibilities were simulated in various operating conditions to build the training dataset. The flexible job shop condition at each monitoring period was described by seven features, such as machine flexibility, machine utilisation, bottleneck status, allowance factor and work in progress. SL models performed better than DRs for optimising mean tardiness and mean flowtime, and the model ensemble was the best-performing algorithm.

Durasević and Jakobović (2022) explored PRS with parallel machines. The work compares different algorithms: NN, DT, k-means, and Naïve Bayes. The candidate DRs were previously generated using Genetic Programming (GP). The dataset was built by simulating episodes of small lengths. After stationary and non-stationary tests, decision trees performed the best in optimising total weighted tardiness.

Real-time rule selection

Regarding RTRS, one can find several works relying on standard Q-Learning (QL), (e.g. Wang and Usher 2005; Wei and Zhao 2004; Chen et al. 2010), which is more suitable for discrete state spaces. To overcome this obstacle, authors discretised continuous variables in several bins. However, the number of bins must was kept low, to reduce the state space to explore, thus these works rely only on one to two features.

Wang and Usher (2005) proposes a QL agent for a single machine problem. The system relies on two state features, the number of jobs in the buffer and the estimation of total lateness, which are discretised in ten bins. There are three reward functions, one for each three of the optimization objectives. The agent was tested in three simulation cases. Each case had a suitable DR, EDD, FIFO, SPT. In the end, the agent was able to select the most suitable DR more frequently in the correct cases.

Wei and Zhao (2004) proposes QL for dynamic JS. The agent relies on a single feature called pressure, which is based on a ratio between lateness and remaining processing time. This feature is discretised in five bins. The RL agent can select between three DRs proposed by the author, which are based on the FIFO and CR DRs. Rewards are lateness based. The improvement brought by the QL agent regarding mean tardiness is very low when compared with the proposed DRs, which may be linked with the small instances of fifteen jobs.

Chen et al. (2010) proposes a QL agent for the multi-objective dynamic JS problem. Three DRs (CR, MOD, SOPT) were filtered from a set of twelve standard DRs. The selection was made according to their performance regarding mean tardiness and mean flowtime criteria and resorting to the Data Envelopment Analysis (DEA) technique. The system relies on a single feature, based on work in progress, which was discretised in 11 bins. The agent does not select DRs directly, but finds the weights used to combine the three elected DRs in a composite DR. The reward has three components, one that concerns the work in progress evolution and two more regarding each one of the optimisation objective. The best model was able to dominate all benchmark DRs.

In order to propose DRSS with a higher number of features, authors started coupling standard QL with clustering as an alternative to binning, (e.g. Yang and Yan 2009; Wang 2020).

Yang and Yan (2009) tackles the RTRS problem in a dynamic JS with QL. The state set is comprised by four features that concern machine workload, slack time, system utilisation and allowance factor. There are three standard DRs available, EDD, SPT, SLACK. The agent receives a reward when a job is completed, which is equal to the tardiness of the job. The proposed QL-based system is effective at minimising mean tardiness, particularly in instances with uncertain processing times.

Wang (2020) proposed a QL agent for RTRS in a JSSP. The agent relied on features such as machine utilisation, machine load, lateness, and allowance factor. However, classic QL is unsuitable for continuous state spaces. The agent could choose between four simple rules: EDD, SPT, FIFO, and SLACK. The reward function consisted of the difference of the optimisation criterion metric between decisions, more precisely, a weighted lateness metric.

As time went by, the field of deep learning evolved rapidly, therefore a deep learning QL version was proposed, in which the Q-function is approximated by a Neural Network. This version is much more suitable for continuous state spaces.

Luo (2020) developed a Deep QL RL agent to select the most appropriate DRs at each decision point for the flexible DJSSP. The condition of the job shop was described by features such as machine utilisation, tardiness, jobs' completion rate, and operations' completion rate. The agent could choose between 6 custom dispatching rules. The reward function was unitary, i.e., if tardiness improved between dispatching

decisions, the agent would be rewarded with *+1*. Otherwise, it would be punished with a *-1*. Luo et al. (2021) extended the previous work by considering multiple objectives, namely machine utilisation and tardiness. In the first step, the agent chooses the best metric to optimise and, in a subsequent moment, the DR that should be used. The proposed RL agent performed better than benchmark DRs in over 80% of the test instances.

Finally, opposing to all previous authors, Mouelhi-Chibani and Pierreval (2010) states that tackling the DRSS with RL is hard due to the temporal credit assignment problem. Furthermore, SL is also challenging, because finding a dataset is not trivial. Thus, he proposes a simulation optimisation technique for the RTRS on a flow shop. An initial NN is established. This NN is not optimized by backpropagation (as standard in Supervised learning), nor by incorporating rewards (as it is mandatory in RL), but through a combination of simulation and simulated annealing (SA). The agent relies on 22 hard-coded features and five standard DRs (EDD, SPT, LPT, FIQ - first in queue - and FIS - first in system). In the end, an improvement of 10% in mean tardiness is observed.

Table 1 outlines the differences between this work and previous DRSS works. It becomes evident that, in our case, a thorough comparison of existing DRSS approaches is attempted, as there are models proposed for both PRS and RTRS. For each approach, several algorithms were considered. In the case of PRS, RF, NN and SVM models were trained for both classification and regression tasks. Note that DT is a prevalent method in PRS, although RF is more powerful. Thus, the latter was preferred. For RTRS, both QL and DQL were implemented, which allows us to assess the impact on performance of discretisation. Finally, the models were tested in a comprehensive array of JS instances, including cases of unexpected events such as a sudden spike in job arrivals.

Furthermore, this work pays attention to every component of a DRSS. Regarding the action set, most instances of Table 1 rely on a few standard DRs (e.g., EDD, SPT, and FIFO). In this work, the performance of a large set of both expert and AI-generated DRs was tested in various conditions. Such assures that the chosen rules of the action set are valid alternatives between themselves, and render superior performance. Regarding the state set, most works define it without any deep analysis of the features involved or rely on large state sets that become a burden to the DRSS. In this work, a correlation analysis is carried out to filter out those features with low added value, ultimately finding a compact and effective state set. To finish, our DRSS comes with a monitoring component called flusher that detects operations that have stayed for too long in the system, pushing them to be processed first.

4. Methodology

As depicted by Figure 1, the DR selection system encompasses four main components, the *state* and *action sets*, the *agent* and the *environment*.

The state set is the list of features that describes the job shop condition at a given time. Over the years, several state features have been proposed in the DRSS literature. In Table 2, those features were grouped into four categories, machine, pending work, due date and arrival-based related features.

The action set consists of the list of DRs that can be chosen by scheduling agents. Successful action sets are not composed necessarily of the most performant rules overall but of the set of best-performing rules for different operating conditions so the DRSS

Table 1. Overview of past literature on the DRSS topic. References are separated by PRS and RTRS. Within each group, sorting is chronological. It highlights the fact that this work includes simultaneously both PRS (classification and regression) and RTRS. Additionally, it becomes evident the care in the feature and DRs selection process when compared with past literature, due to the few elements that remained from the original sets, e.g. 3 of (78) DRs.

References	Problem	Optimisation Objective	State Set Size	Action set Size	Type of DRs	Models	PRS Models Class.	PRS Models Regr.	RTRS	Non-stationary
Park et al. (1997)	Flexible Flow Shop	Tardiness	3 (6)	4	Elementary	DT	✓			✓
Min and Yih (2003)	Flexible Flow Shop	Flowtime, Slack, Remaining Time	5	7	Elementary	NN	✓			
Liu et al. (2005)	Flexible Job Shop	Throughput	26	7	Elementary	SVM	✓			
Metan et al. (2010)	Job Shop	Tardiness	(26)	3, 4	Elementary	DT	✓			✓
Choi et al. (2011)	Hybrid Flow Shop	Throughput, Flowtime, Tardiness, Number of Tardy	4	14	Elementary	DT	✓			✓
Shiue et al. (2012)	Flexible Job Shop	Throughtput, Cycle Time, Number of Tardy	(30)	5	Elementary	DT, NN, SVM, Ensemble	✓			✓
Priore et al. (2018)	Flexible Job Shop	Tardiness, Flowtime	7	4+4	Elementary	DT, k-nn, SVM, NN, Ensemble	✓			✓
Durasević and Jakobović (2022)	Parallel Machines	Tardiness	2-6	10-50	Composite (AI-generated)	DT, k-nn, Naive Bayes, NN	✓			✓
Wei and Zhao (2004)	Job Shop	Lateness-Remaining Time	1	3	Composite (Custom)	QL			✓	
Wang and Usher (2005)	Single Machine	Lateness, Number of Tardy	2	4	Elementary	QL			✓	✓
Yang and Yan (2009)	Job Shop	Tardiness	4	3	Elementary	QL			✓	✓
Chen et al. (2010)	Job Shop	Tardiness, Flowtime	1	3	Elementary	QL			✓	
Mouelhi-Chibani and Pierreval (2010)	Flow Shop	Tardiness	22	5	Elementary	NN+SA			✓	✓
Luo (2020)	Flexible Job Shop	Tardiness	7	6	Composite (Custom)	DQL			✓	
Wang (2020)	Job Shop	Earliness-Tardiness	4	4	Elementary	QL			✓	
Our work	Job Shop	Tardiness	2, 3 (13)	3 (78)	Elementary and Composite	RF, NN, SVM QL, DQL	✓	✓	✓	✓

can learn to select the most appropriate DR at a convenient time. The goal is to find a complementary action set.

The agent is the model responsible for the mapping between job shop states and suitable DRs. Agent training differs for PRS and RTRS, as explained in detail in the following subsections.

The environment is the JS simulator. It is responsible for three main tasks. Firstly, generating dynamic job arrivals, according to some predefined parameters. Secondly, allocating jobs to machines, whenever these become idle. Such is done by computing a score for each operation in the idle machine queue, according to the active DR, and then selecting the operation with the best score to be processed next. Finally, selecting DRs over time as conditions change. In PRS, every time a period ends, the simulator summons the agent, which in turn assesses the state and selects the DR for the next period. In RTRS, the agent is summoned every time a machine becomes idle.

Additionally, a fifth component was added, the flusher, that prevents orders from staying abnormally long time in the job shop. Such happens because DRs are myopic, particularly the most simple ones. For instance, some rules are *processing time* oriented, while others are *slack* oriented (Geiger et al. 2006). This DRs drawback may leave some operations *stuck* in the JS, particularly operations with tail distribution values. The flusher was implemented to fight this phenomenon. It operates as follows: when an operation remains longer than the established time limit, the DR suggested by the agent is overruled, and the *stuck* operations are scheduled first.

4.1. *Agents for periodic rule selection*

The agents for periodic rule selection (PRS) were trained via SL, namely Random Forests (RFs), Neural Networks (NNs) and Support Vector Machines (SVMs). Regardless of the algorithm, SL requires datasets to train the models. The datasets were

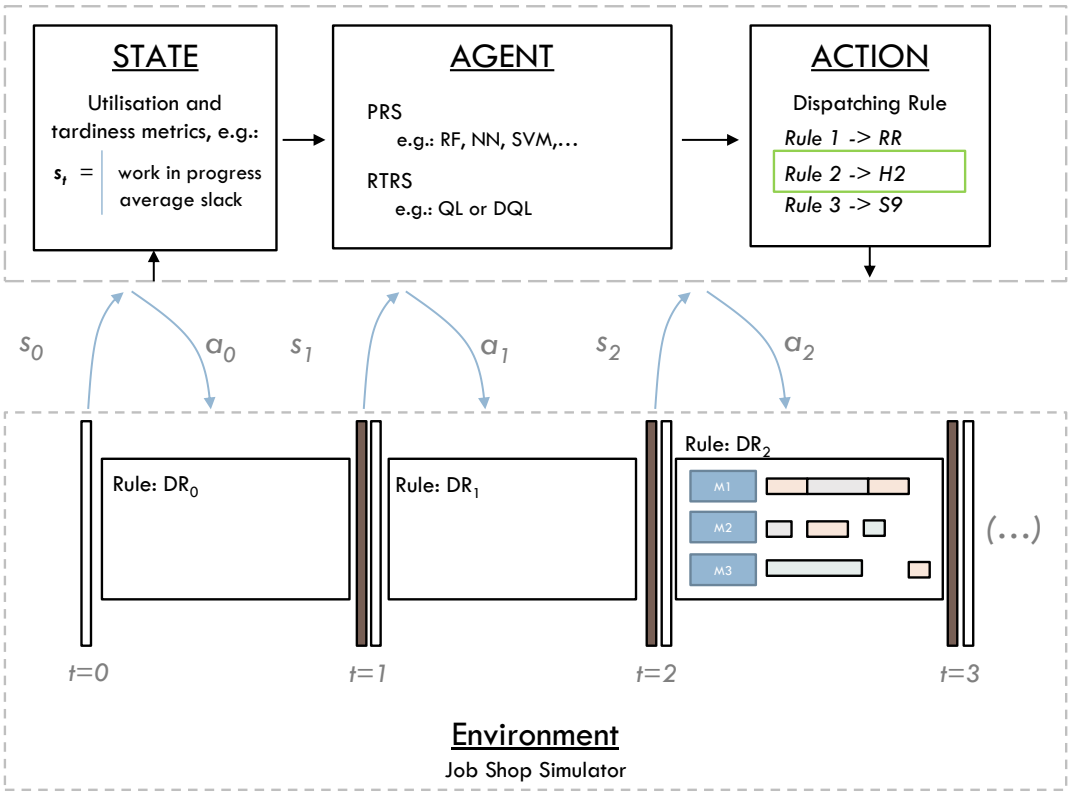


Figure 1. DRSS main components: agent, state, action (DR) and environment (job shop simulator). The agent is responsible for assessing the state at different points in time. Then, according to the observed conditions, it suggests a suitable DR to be deployed in the job shop simulator. Finally, it waits until the following DR selection moment to restart the process.

built resorting to a technique called multipass simulation (Wu and Wysk 1989). This technique splits long episodes into small periods, as Figure 2 depicts. Then, all rules are tested in a given period from the same starting point (at the beginning of the period), resulting in different levels of tardiness. The best-performing rule in that period (i.e., the one that results in the lowest tardiness, as described by Equation 2) can be used as a label for a classification algorithm.

$$Selected\ DR = \underset{DR}{argmin}\left(\Delta Tard_{DR,t}\right) \tag{2}$$

Alternatively, a regression method can be used, where a label is determined for each DR and corresponds to the tardiness resulting from applying that DR. This tardiness measure can be either the absolute or relative value. In our case, the label, $Label_{DR,t}$, is computed as the relative difference between the increment in cumulative tardiness (of all jobs since the beginning of the episode) provided by the DR during this period, $\Delta Tard_{DR,t}$, and the minimum increment in cumulative tardiness recorded by all DRs for that period, $\Delta Tard_{min,t}$, as stated by Equation 3.

$$Label_{DR} = \frac{\Delta Tard_{DR,t} - \Delta Tard_{min,t}}{\Delta Tard_{min,t}} \tag{3}$$

Table 2. Over the years, several state features have been proposed. However, many are related, and thus belong to one of the four feature groups. Such suggest that feature selection may have a strong impact.

Variable type	Variable	Source
Pending work related	J, number of pending jobs	Shiue et al. (2012)
	PT, processing times of pending operations	Shiue et al. (2012)
	RPT, remaining processing times of pending jobs	Shiue et al. (2012)
	TPT, total processing times of pending jobs	Shiue et al. (2012)
	WINQ, work in the next queue	Shiue et al. (2012)
Due date related	SLK, slack of pending jobs	Shiue et al. (2012)
	TARD, tardiness of pending jobs	Shiue et al. (2012)
	ALLOW, allowance factor	Shiue et al. (2012)
	LAT, lateness of pending jobs	Wei and Zhao (2004)
	EARL, earliness of pending jobs	Wang (2020)
Machine related	U, machine utilisation	Luo (2020)
	A, machine availability	Park et al. (1997)
Arrival rate related	DIST, arrival rate distribution	Durasević et al. (2023)

After evaluating all DRs in a given period, we proceed to the next period. The state at the beginning of this new period can be determined by applying any DRs. Ideally, we should explore the full space of sequences of DRs. However, depending on the number of periods, that can be impractical. Therefore, the multipass approach randomly selects and applies one of the DRs. Note that a random selection, rather than selecting the best DR, promotes a wide range of possible system states.

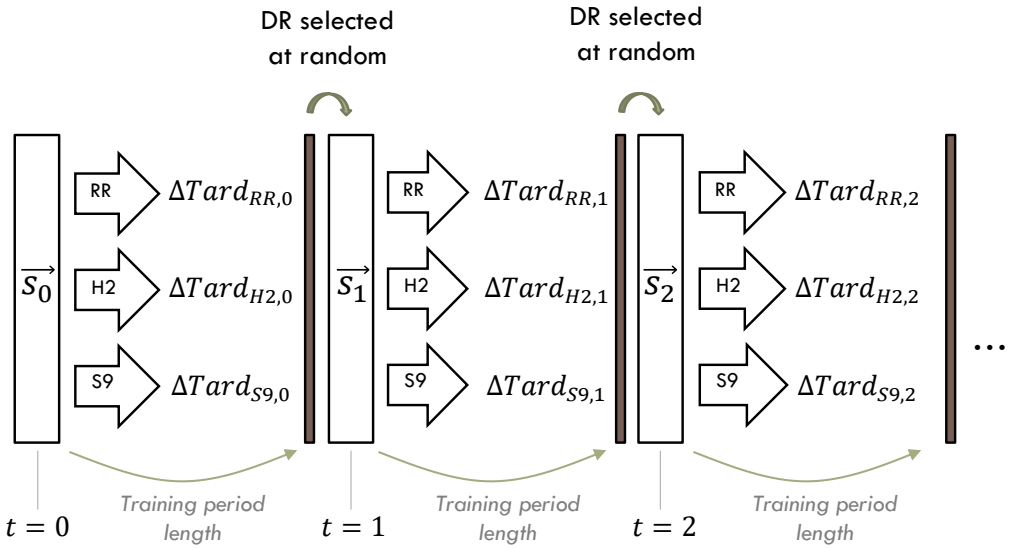


Figure 2. Overview of the training data generation process, which is based on the multipass simulation technique (Wu and Wysk 1989). Briefly, long scheduling episodes are split into several consecutive shorter periods. Then, the behaviour of each DR is evaluated in each of the periods state. The dataset is built by registering the state in the beginning of the period and the performance of each DR at the end. The system advances for the next period by selecting a DR at random.

According to the literature, the length of the training period is a key parameter. It must be long enough to measure the impact of the DR. However, it can not take too long to capture changes in job shop conditions.

4.1.1. Models for periodic rule selection

Different training datasets were built depending on the window length used to split episodes, more precisely, lengths of 50, 100, 200, 400, and 800 jobs. Therefore, it was necessary to train RFs, NNs and SVMs models for each one of the datasets for both classification and regression.

The RFs were composed of 10 trees, with a maximum depth of 6. The NNs had three hidden layers of 4 neurons each. Sigmoid and ReLu were the activation functions for classification and regression, respectively. The kernel of the SVM models were radial basis functions. Due to the stochastic nature of RF and NN training, alternative models were generated resorting to 5 seeds. SVM training is deterministic.

Overall, considering all the types of algorithms, learning tasks, datasets, states sets, and different seeds, 450 PRS models were generated, as depicted by Table 3. Note that for regression, it is necessary to train a model for each DR of the action set. For simplicity sake, the DR-specific models are considered sub-models as they must used together.

Table 3. Number of PRS models.

Learning task	Classification			Regression		
Algorithm type	RF	NN	SVM	RF	NN	SVM
Training seeds	5	5	5	5	5	5
State sets		3			3	
Training datasets		5			5	
Number of models	75	75	75	75	75	75

4.2. *Agents for real-time rule selection*

For RTRS, the agents were trained via RL. In RL, no datasets are necessary as agents improve by interacting with the environment, the job shop simulator in our case. At each decision point t, i.e., every time a machine becomes idle, the agent assesses the state, s_t, and selects a DR, a_t, according to the agent's policy. The policy is the model responsible for the mapping between states and actions. The impact of an agent decision comes in the subsequent decision point, in the form of a reward, r_{t+1}, which is set by the reward function.

QL was the RL method adopted to train the agents. The QL policy is called Q-function. It is responsible for estimating the expected future reward of each state-action pair, $Q(s_t, a_t)$. The Q-function improves by incorporating rewards over time. There is a Q-function update at each decision point, in which state-action pairs are affected by their respective rewards, as shown by Equation 4. The learning rate, α, defines how much an update modifies the existing $Q(s_t, a_t)$ value. The discount factor, γ, defines the weight that subsequent decision points take in the current update.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \cdot \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \tag{4}$$

It is possible to discriminate two types of QL, standard QL and deep QL. In standard QL, the Q-function takes the form of a lookup table, thus commonly called Q-table. It is suitable for discrete state spaces. Therefore, if used in problems with continuous state spaces, two solutions exist: state-space discretisation or deep QL. In deep QL, the Q-

function is approximated by an NN, and no discretisation is necessary. Due to DQL's continuous nature, the Q-function update presented in Equation 4 is not suitable. Instead, the procedure depicted by Algorithm 1 is followed. Briefly, transitions (i.e., $s_t, a_t, r_{t+1}, s_{t+1}$ tuples) are stored in a replay buffer. At each decision point, a mini-batch of transitions is randomly selected from the replay buffer, which is used to train the network using gradient descent.

From Algorithm 1, it is possible to observe the presence of two NN-approximated Q-functions, the online network, $Q(\theta)$, and the target network, $\hat{Q}(\theta^-)$. This technique, called Double Deep Q-Network (van Hasselt et al. 2015), avoids having the online Q-network evaluating its own actions, which could compound estimation errors, thus the target Q-network is introduced. The weights of the networks must be synchronised periodically. In this work, weight synchronisation occurs in the end of a scheduling episode.

Algorithm 1 Q-function update, Deep QL

Randomly sample a set of transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ from Replay Buffer

Set $y_t = \begin{cases} r_{t+1} \text{ , in the last step of the episode} \\ r_{t+1} + \gamma \hat{Q}\left(s_{t+1}, \text{argmax}_{a'} Q\left(s_{j+1}, a', \theta\right); \theta^-\right) \text{ , in all previous steps} \end{cases}$

Perform a gradient descent step on $\left(y_t - Q\left(s, a_t, \theta\right)\right)^2$ with respect to the network parameters θ

RL agents, particularly in the early stages of training, should thoroughly explore the state-action space. As the agent gains experience, it should rely more on its own decisions (exploitation) and less on exploration. The epsilon-greedy exploration strategy become popular to manage the exploration-exploitation balance. Under this strategy, the exploration rate, which controls the percentage of random actions, gradually decreases during training.

4.2.1. Models for real-time rule selection

In order to apply standard QL to a continuous state space problem, each variable was discretised in five bins. Bins' superior and inferior limits were set based on training data and are present in Table 4.

Table 4. QL binning thresholds for each state feature. Limits were set based on the log transformation of the training data.

	Processing times of pending operations		Tardiness of pending jobs		Earliness of pending jobs	
Bin	Inferior limit	Superior limit	Inferior limit	Superior limit	Inferior limit	Superior limit
1	<1096		<12		<6633	
2	1096	1807	12	180	6633	14764
3	1807	2980	180	2440	14764	32859
4	2980	4914	2440	32859	32859	73129
5	>4914		>32859		>73129	

In alternative to QL, which requires state discretisation, DQL can be applied instead. It approximates the Q-function by an NN. The input layer of the tested NNs receives the state and the chosen action at time *t*. Thus, the size of the input changed according to the state set, four for SS_1 and SS_2, and five for SS_3. Note that the

action is encoded as a dummy variable, so two neurons are necessary. The NNs had 3 hidden layers with the same size as the input layer. The output of the NN was the estimation of the expected future reward, $Q(s_t, a_t)$.

Regarding the Q-function updates, both QL and DQL share the same reward function. According to the literature, the reward function should follow the optimisation criterion (Wei and Zhao 2004). Therefore, the reward function is tardiness driven: if tardiness improves, the agent is encouraged; otherwise, it is punished, see Equation 5.

$$r(t) = \begin{cases} +1, \text{if estimated tardiness improves} \\ -1, \text{if estimated tardiness worsens} \\ \text{if tardiness is unchanged, untie with slack} \end{cases} \tag{5}$$

Moreover, to perform a Q-function update it is necessary to set up the learning rate and discount factor parameters. In QL, the learning rate took the values 0.2, 0.3, 0.4. For the discount factor, the following levels were tested: 0.7, 0.8, 0.9. In DQL, the discount factor, γ, took the values: 0.80, 0.85, 0.90. The learning rate, which in DQL is necessary for the back-propagation process, took the following values: 0.001, 0.005, 0.01. Overall, 45 independent runs were conducted for each QL and DQL, as there are nine possible learning rate and discount factor combinations and five initial random policies.

An epsilon-greedy strategy was implemented to manage exploration-exploitation during learning. The exploration rate started high, at 90%, and linearly decreased to 10%. The rate decreased every time a training cycle finished, i.e., every time the agent went through the training instances. The decrease is 0.004 in DQL and 0.0004 in QL. The rate is lower in QL as it performs a larger number of training cycles due to a faster Q-function update procedure. The agent iterates on the training instances until the time limit of two days is reached.

5. Computational experiments

This section details the instances that were used for system design, and for model training, validation, testing and stress behaviour. Furthermore, it shows the tests that support the design decisions for the main components of the DRSS, namely, state set, action set and flusher components. Finally, the results of the models in those instances.

5.1. *Experiments Design*

The JS instances were composed of 20 machines. Job arrival follows a Poisson distribution: Job Arrival $\sim Poisson\left(\lambda = \frac{M \cdot u}{\overline{n} \cdot \overline{p}}\right)$, where M stands for the number of machines, the u parameter defines the utilisation level, $\overline{n}$ is the average number of operations per job, and $\overline{p}$ is the average processing time. Irrespective of the utilisation level, the simulation stops when 2500 jobs are completed. The first 500 jobs are not included in tardiness computation as the JS takes time to reach a stable operational level. Jobs' due dates are defined according to the *Total Work Content* rule (Baker 1984): $d_j = a \cdot \sum_{o=1}^{O_j} p_{jo}$, where the a parameter, the allowance factor, defines the amount of time available to process jobs. Jobs' routes vary in size from 2 to 14 operations. Operations' processing times are given according to a uniform distribution that ranges

from 1 to *ptMax*.

Each tuple *u*, *a*, *ptMax* comprises a scenario *s* from the set o scenarios *S*. Notice, from Table 5, how the number of scenarios keeps increasing from *Train* to *Validation* and from *Validation* to *Test* sets due to the desire to assess model robustness and generalisation capabilities. An instance is an individual episode from any given scenario. For all *Train*, *Validation*, and *Test* sets there are 100 episodes per scenario, except in RL, as training is much lengthier.

Table 5. The JS parameters. In order to expose the DRSS models to a wide range of load conditions, several parameters (e.g. the utilisation level, allowance factor and average processing times) can assume different levels. Notice how the number of levels increases as the model evaluation goes from train to validation and from validation to test. Such enables to evaluate the model's generalisation capabilities.

	Parameter	Train	Validation	Test
M	Number of machines	20		
J	Number of jobs (warm-up)	2500 (500)		
O_j	Number of operations per job	U(2,14)		
p_{jo}	Processing time of operations	U(1,ptMax): ptMax={50,100}		ptMax={50,100,200}
u	Utilisation level	0.85, 0.90, 0.95	0.83, 0.87, 0.93, 0.96	0.80, 0.85, 0.90, 0.95, 0.97
a	Allowance factor	3, 4, 6	2.5, 3.5, 5, 7	2, 3, 4, 6, 8
S	Number of scenarios	18	32	75
nRep	Number of episodes per scenario	100 for SL 25 for QL 5 for DQL	100	100

In a deeper effort to test the DRSS against increased uncertainty, the base instances were modified to include stress events. Such was achieved by altering the utilisation and allowance in the middle part of episodes, similar to Durasević et al. (2023). Two different patterns were explored to mimic different stress scenarios that might occur in real life, as depicted by Figure 3. A stress scenario can be described by the tuple *u*, *a*, *ptMaxFor* and u_s, where u_s stands for the *utilisation level* in the middle part of the episode. There are 100 replications per scenario.

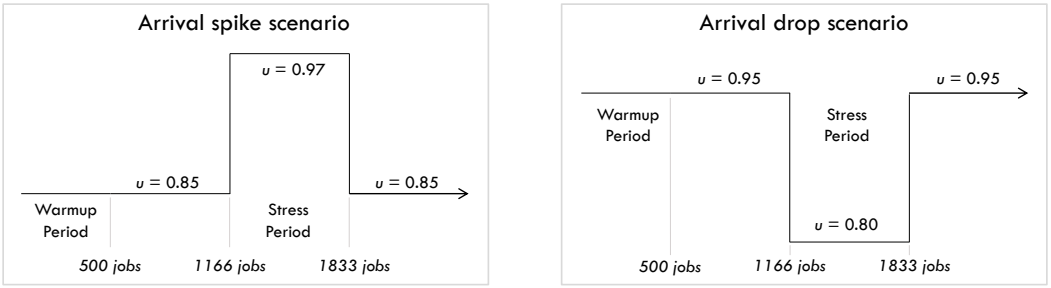


Figure 3. Non-stationary instances: The pattern on the left aims to simulate a sudden arrival of urgent jobs; The pattern on the right represents a moment in which the job shop has a steep drop in job arrivals. For each type of stress period, a spike or a drop, two versions were tested: one with high urgency (a=3), and the other with low urgency (a=6).

Different scenarios render tardiness values of different magnitudes. Therefore, when assessing the performance of the models, one cannot rely on tardiness values directly, as the high tardiness scenarios (e.g.: u=0.97 and a=2) would have greater importance than lower tardiness scenarios (e.g.: u=0.80 and a=8). Consequently, models are compared resorting to a normalised tardiness score NT.

The first step to compute NT, is to compute the performance of the model for each scenario, NT_s. Such is achieved by computing the model's mean tardiness, $\sum_{r=1}^{r=nRep} T_{s,r}$, for each scenario divided by the corresponding reference value, RT_s,

see Equation 6. The reference for each scenario is the average performance of the most suitable DR for that specific scenario. The performance score, NT is the geometric mean of the normalised values across all scenarios, as expressed by Equation 7.

$$NT_s = \frac{\frac{1}{nRep}\sum_{r=1}^{r=nRep} T_{s,r}}{RT_s} \tag{6}$$

$$NT = \left(\prod_{s}^{S} NT_s\right)^{\frac{1}{|S|}} \tag{7}$$

A score equal to one means that the model is equivalent, on average terms, to use the best rules in each one of the scenarios. Scores below one are possible for those models that outperform the reference values.

5.2. *Results*

This section shows the results of the various tests carried to define each of the system components. Afterwards, the performance of the several trained PRS and RTRS models is assessed in a wide array of scenarios, including stress instances.

5.2.1. State set definition

The first step regarding feature selection, consisted in removing some variables according to the following assumptions:

- Firstly, features should have a global scope. A DR is to be chosen for the entire job shop, so the decision should not rely on information from a single queue (Rajabinasab and Mansour 2011). Consequently, the DRSS should not depend on a variable such as *work in the next queue*;
- Information about the future is more important than past information. Therefore, features such as *machine utilisation*, *machine availability*, *total processing times of pending jobs*, *arrival rate distribution*, are deemed less relevant;
- Two main drivers impact DR performance in job shops, utilisation and allowance (Baker 1984). Thus, at least a variable for each dimension is necessary.

In order to find the most interesting state sets, variables were grouped in pending-work and allowance related variables. Subsequently, correlations were computed between variables from the same category, as shown by Table 6.

In the *pending work-related features* group, high correlation values are observed, therefore a single feature is deemed fit to model the utilisation level of the JS. The variable *processing times of pending operations* was the selected one as it is the most common in the literature. To model the allowance level of the job shop, two features were chosen: the *tardiness of pending jobs* is selected, as well as the *earliness of pending jobs*. These features were chosen due to low correlation between them, 0.17. No other feature was deemed necessary, as the *tardiness of pending jobs* feature displays a high correlation with the remaining variables. All in all, the proposed DRSS rely on three possible state sets:

Table 6. Correlation matrices. Pending work-related features on the left (a), allowance-related features to the right (b). Within each group, strong correlations (>0.70) are often observed, suggesting that most variables behave similarly and are, consequently, inessential.

(a)

	J	**PT**	**RPT**
J	-	0.69	0.75
PT		-	0.92
RPT			-

(b)

	SLK	**TARD**	**ALLOW**	**LAT**	**EARL**
SLK	-	0.91	0.99	1.00	0.56
TARD		-	0.87	0.91	0.17
ALLOW			-	0.99	0.62
LAT				-	0.56
EARL					-

- SS_1: *Processing times of pending operations & tardiness of pending jobs*;
- SS_2: *Processing times of pending operations & earliness of pending jobs*;
- SS_3: *Processing times of pending operations & tardiness of pending jobs & earliness of pending jobs*;

5.2.2. Action set definition

To define the action set every DR was tested in the train instances. It became clear that each DR has preferential operating conditions. In fact, three zones emerged in the utilisation-allowance state space, as depicted by Table 7.

- low allowance, dominated by rules S9, S8 and SHA3;
- high allowance and high utilisation, dominated by the RR rule;
- high allowance and low utilisation, the most scattered zone in space, where rules H1, H2, H4, OZ1, AND2 (CR+SPT) and PL3 emerged.

Table 7. Best performing DR for different utilisation u, allowance a, and processing time $ptMax$ levels. It becomes clear that some DRs, e.g. S9, S8 and SHA3, are suitable for tight due dates, while others perform better when due dates are looser, e.g. H1, H2, H4, PL3, AND2, OZ1 for low utilisation and RR for high utilisation instances. The action set should include DRs from each one of the zones.

(a) ptMax=50

$a \setminus u$	0.8	0.85	0.9	0.95	0.97
2	SHA3	SHA3	SHA3	S9	S9
3	S9	S8	S9	S9	S9
4	AND2	S9	S9	S9	S9
6	H2	H4	RR	RR	RR
8	PL3	H2	H1	RR	RR

(b) ptMax=100

$a \setminus u$	0.8	0.85	0.9	0.95	0.97
2	SHA3	SHA3	SHA3	S9	S9
3	S9	S8	S9	S9	S9
4	H4	S9	S9	S9	S9
6	OZ1	H2	RR	RR	RR
8	OZ1	H2	AND2	RR	RR

The goal is to find an action set with performant rules for different shop conditions. Thus, several candidate action sets were defined by selecting one DR from each zone. There are 18 possible combinations in total. The best performing one was the S9, RR, H2 set, as depicted by Table 8. The proposed DRSS relied on this set.

5.2.3. Flusher definition

In order to define the thresholds at which the flusher is activated, the RR, S9, and H2 DRs were simulated in the training instances. From Figure 4, it is possible to observe that the best tardiness scores were obtained when no operation remained longer in the system than 7500 u.t. for instances with ptMax = 50 or 14000 u.t. for ptMax = 100. For ptMax = 200 instances, the threshold of 28000 u. t. was extrapolated.

Table 8. Candidate DRs combinations' performance. The S9 + RR + H2 action set displayed the highest potential for tardiness reduction, thus it was adopted in the proposed DRSS.

Possible DR Combinations				Performance Score
1	S9	RR	H2	1.036
2	S9	RR	H4	1.039
3	S9	RR	AND2 (CR+SPT)	1.039
4	S9	RR	H1	1.055
5	S9	RR	OZ1	1.064
6	SHA3	RR	H2	1.065
7	SHA3	RR	H4	1.066
8	SHA3	RR	AND2 (CR+SPT)	1.066
9	S8	RR	H2	1.069
10	S8	RR	H4	1.070
11	S8	RR	AND2 (CR+SPT)	1.070
12	SHA3	RR	H1	1.084
13	S8	RR	H1	1.088
14	SHA3	RR	OZ1	1.097
15	S8	RR	OZ1	1.101
16	S9	RR	PL3	1.156
17	SHA3	RR	PL3	1.195
18	S8	RR	PL3	1.199

5.2.4. Supervised learning models for Periodic Rule Selection

First of all, the impact of the state set on the performance of the PRS models is analysed resorting to Figure 5. It presents the performance of the PRS models in the validation instances grouped by algorithm, learning task and state set, and shows that models that rely on SS_2 render worst average performance and higher variability than SS_1 and SS_3 based models. Moreover, it is also observable a performance decline, on average, when going from classification to regression.

As seen previously, there are 450 distinct PRS models. However, each PRS model can be used with different DR selection frequencies, raising the total number of possible combinations to 1350 datapoints. In fact, the length of the training periods, as well as the frequency at which the system assesses its state to select an updated DR, deeply impact the PRS effectiveness. According to Table 9, the best results were found when the training periods were longer (400 or 800 jobs) and the DR selection frequency was shorter (w50 or w100). The larger training periods allow for a better estimation of the impact of DRs and the shorter selection periods allow for a more reactive behaviour.

Table 9. Tardiness score of the best performing PRS model for each decision frequency and training period length combination, according to validation performance. The three best performances are underlined.

		Decision frequency				
		w50	w100	w200	w400	w800
Train window length	w50	1.002				
	w100	1.010	1.020			
	w200	1.000	1.010	1.021		
	w400	<u>0.994</u>	1.001	1.012	1.024	
	w800	<u>0.989</u>	<u>0.995</u>	1.003	1.009	1.032

Finally, Table 10 shows the performance of the best train window length, DR selec-



Figure 4. Tardiness performance of each DR from the action set was obtained for the training instances. For each DR-instance combination, the episode arrival rate and maximum time a job remained in the system was recorded. These two variables were used for the *xx* and *yy* axis variables, respectively. On top, the data from instances with ptMax=50. At the bottom, data from instances with ptMax=100. The plots show that when using the most desirable DR on any given instance, jobs tend to stay in the system for shorter periods. Based on the plots, the thresholds for flusher activation could be easily estimated, and are represented by a red horizontal line.

tion frequency and state set combination for each algorithm-task pair in validation and test instances. According to results, RF-Reg, NN-Class and SVMs present very good performance in validation and test instances, showing good generalisation capabilities. The tardiness score is around 1.00, which means the agent can effectively adapt to different load conditions. RF-Class displays a generalisation problem. In spite of a good tardiness score in validation, at par with the best models, in test instances performance deteriorates substantially. Finally, the NN-Reg models' validation performance is the worst registered among DRSSs and they don't generalise well. Irrespective of the algorithm-task combination, the proposed PRS models show great improvement over using a single DR. Overall, SVM-Reg performed better than other combinations.

Table 10 reinforces previously mentioned conclusions. To start, the best models prefer longer training periods (400 or 800 jobs) and shorter DR selection frequencies (w50). Furthermore, no algorithm-task combination relies on SS_2.

5.2.5. RL learning agents for Real Time Rule Selection

The performance of the RTRS models for the validation instances can be found in Figure 6. Results are grouped by algorithm and state set. Similar to the results of the supervised learning models, the SS_2 based models present worst performance on average and also higher variability than SS_1. SS_3 based models behave differently when used by QL or DQL. SS_3 based QL models perform worst on average than

both SS_1 and SS_2. SS_3 based DQL models clearly outperform SS_2, and are able to outperform SS_1 slightly.

From Figure 6, it is also observable that DQL performs clearly better than QL in validation instances. In fact, the best models overall are also DQL. Such is expected as the state variables are continuous, which QL can not tackle directly.

Table 10 shows the performance of the five best RTRS models for both QL and DQL, chosen according to their performance in validation instances. As stated previously, QL performance is inferior to DQL's in both validation and test instances. Furthermore, for both QL and DQL, there is a steep performance decrease of at least 0.05 when going from validation to test. DQL, in spite of outperforming SVM models in validation instances, performs worst than SVM (0.03 increase), as it struggles to generalise. Nonetheless, RTRS models perform significantly better than using a single DR.

Table 10. Performance of the DRs, PRSs and RTRSs for both validation and test instances. In PRS, only the performance of the best combination of parameters (train window length, DR selection frequency and state set) for each algorithm-task is shown. In RTRS, only the performance of the best state set is present for each RL algorithm. The best-performing models are the RF-Reg, NN-Class and SVM-based PRS models. Irrespective of the algorithm, all proposed DRSSs perform substantially better than DRs.

Algorithm	**Task**	**Train window length**	**DR selection frequency**	**State set**	**Tardiness score**			
					Validation		**Test**	
					Average	**WCS**	**Average**	**WCS**
DR	RR	-	-	-	1.146	-	1.219	-
DR	H2	-	-	-	1.624	-	1.509	-
DR	S9	-	-	-	<u>1.102</u>	-	<u>1.213</u>	-
RF	Class.	800	50	SS_1	0.995	1.002	1.060	1.065
	Reg.	800	50	SS_3	0.998	1.005	1.000	1.005
NN	Class.	800	50	SS_1	0.989	0.991	0.997	0.998
	Reg.	100	50	SS_1	1.023	1.054	1.039	1.086
SVM	Class.	400	50	SS_3	0.994	0.997	0.997	0.998
	Reg.	800	50	SS_3	<u>0.992</u>	0.993	<u>0.995</u>	0.995
RL	QL	-	-	SS_1	1.007	1.014	1.060	1.091
	DQL	-	-	SS_3	<u>0.981</u>	0.984	<u>1.034</u>	1.068

Overall, the proposed DRSSs perform significantly better than a single DR approach. However, *How can these systems outperform DRs?* As it is evident from Figure 7, the models suggest the variable H2 mainly when the *tardiness of pending jobs* is low, while the S9 rule is preferred for higher values of the variable.

5.2.6. Performance in non-stationary instances

The performance of the best DRSSs (those present in Table 10) in the non-stationary instances is depicted in Table 11. PRS and RTRS systems outperform the DRs, i.e., present tardiness scores lower than 1.022. In fact, some models outperform DRs in the majority of the tested patterns (e.g.: RF-Reg, NN-class, and DQL). When the urgency is high, the performance of the models is close to 1.000, as the S9 rule is suitable for the vast majority of the time. The DQL model was the best overall (tardiness score = 0.988).

Figure 8 shows both PRS and RTRS models select rules over time when stress situations occur. One can see, on the left, that the S9 rule, which is suitable for high utilisation scenarios, becomes active shortly after a job arrival spike, as expected. Conversely, the right side of the figure shows that when there is a steep job arrival drop, the H2 rule comes into effect after some time. Finally, RTRS leads to a much higher number of rule changes, usually in the hundreds, while PRS is more stable.

Table 11. Performance of the best models in non-stationary instances

Model	Arrival spike (low urgency)	Arrival spike (urgent jobs)	Arrival drop (low urgency)	Arrival drop (urgent jobs)	Average
RR	**1.000**	1.117	**1.000**	1.138	1.062
H2	1.246	1.909	1.215	1.972	1.545
S9	1.041	**1.000**	1.049	**1.000**	**1.022**
RF-class	1.004	1.004	0.999	0.995	1.000
RF-reg	0.999	1.000	0.975	0.995	0.992
NN-class	0.998	0.998	0.973	0.992	0.990
NN-reg	1.020	1.034	0.991	1.012	1.014
SVM-class	1.006	1.000	0.971	0.994	0.993
SVM-reg	1.008	0.998	0.975	0.992	0.993
QL	0.980	1.019	0.961	1.012	0.993
DQL	0.989	1.002	0.967	0.993	0.988

6. Conclusion

DRs present varying performance according to utilisation and job urgency conditions. Moreover, in real-world settings, conditions change pretty rapidly due to uncertainty. Therefore, DRs quickly become inadequate for the environment they are deployed. All in all, this work mitigates that problem by proposing systems that select the most suitable DR as conditions change in dynamic job shops.

According to the results, it is possible to assess that the proposed selection systems can significantly reduce tardiness when compared to state-of-the-art DRs, the improvement lies around 20%. These gains are possible due to a complementary action set and a representative state set, enabling the system to pivot swiftly from low-load DRs to high-load DRs.

In previous works, two types of selection systems were proposed, PRS and RTRS, but it is unclear which is the most advantageous approach. Tests reveal that both types of systems outperform DRs. Additionally, they perform comparatively between each other, showing that both types of systems can effectively learn under which DR should be used. PRS presents a slight advantage in stationary conditions, while RTRS was the best approach in non-stationary conditions.

Although effective, the proposed systems are limited to the performance of the DRs. Consequently, recent literature, mainly based on RL, aims to dispatch jobs directly and not via DRs. It would be interesting to explore how these emerging approaches compare with the rule selection systems proposed in this work.

References

Ahn, G. and Hur, S. (2020). Clustering and dispatching rule selection framework for batch scheduling. *Mathematics 2020, Vol. 8, Page 80*, 8(1):80.

Anderson, E. J. and Nyirenda, J. C. (1990). Two new rules to minimize tardiness in a job shop. *International Journal of Production Research*, 28(12):2277–2292.

Baker, K. R. (1984). Sequencing rules and due-date assignments in a job shop. *Management Science*, 30(9):1093–1104.

Braune, R., Benda, F., Doerner, K. F., and Hartl, R. F. (2022). A genetic programming learning approach to generate dispatching rules for flexible shop scheduling problems. *International Journal of Production Economics*, 243.

Chen, X., Hao, X., Lin, H. W., and Murata, T. (2010). Rule driven multi objective dynamic

scheduling by data envelopment analysis and reinforcement learning. *2010 IEEE International Conference on Automation and Logistics, ICAL 2010*, pages 396–401.

Choi, H.-S., Kim, J.-S., and Lee, D.-H. (2011). Real-time scheduling for reentrant hybrid flow shops: A decision tree based mechanism and its application to a tft-lcd line. *Expert Systems with Applications*, 38(4):3514–3521.

Durasević, M., Gil-Gala, F. J., Planinić, L., and Jakobović, D. (2023). Collaboration methods for ensembles of dispatching rules for the dynamic unrelated machines environment. *Engineering Applications of Artificial Intelligence*, 122:106096.

Durasević, M. and Jakobović, D. (2022). Selection of dispatching rules evolved by genetic programming in dynamic unrelated machines scheduling based on problem characteristics. *Journal of Computational Science*, 61.

Ferreira, C., Figueira, G., and Amorim, P. (2022). Effective and interpretable dispatching rules for dynamic job shops via guided empirical learning. *Omega*, 111:102643.

Geiger, C. D., Uzsoy, R., and Aytuğ, H. (2006). Rapid modeling and discovery of priority dispatching rules: An autonomous learning approach. *Journal of Scheduling 2006 9:1*, 9(1):7–34.

Golmakani, H. R. and Birjandi, A. R. (2013). A two-phase algorithm for multiple-route job shop scheduling problem subject to makespan. *The International Journal of Advanced Manufacturing Technology*, 67(1-4):203–216.

Holthaus, O. and Rajendran, C. (2000). Efficient jobshop dispatching rules: Further developments. *Production Planning & Control*, 11:171–178.

Hunt, R., Johnston, M., and Zhang, M. (2016). Evolving dispatching rules with greater understandability for dynamic job shop scheduling. *School of Engineering and Computer Science, Victoria University of Wellington*.

Khajeh, S. A., Saberikamarposhti, M., and Rahmani, A. M. (2022). Real-time scheduling in iot applications: A systematic review. *Sensors*, 23(1):232.

Liu, Y.-H., Huang, H.-P., and Lin, Y.-S. (2005). Dynamic scheduling of flexible manufacturing system using support vector machines. pages 387–392. IEEE.

Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91:106208.

Luo, S., Zhang, L., and Fan, Y. (2021). Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Computers & Industrial Engineering*, 159:107489.

Metan, G., Sabuncuoglu, I., and Pierreval, H. (2010). Real time selection of scheduling rules and knowledge extraction via dynamically controlled data mining. *International Journal of Production Research*, 48(23):6909–6938.

Min, H.-S. and Yih, Y. (2003). Selection of dispatching rules on multiple dispatching decision points in real-time scheduling of a semiconductor wafer fabrication system. *International Journal of Production Research*, 41(16):3921–3941.

Mouelhi-Chibani, W. and Pierreval, H. (2010). Training a neural network to select dispatching rules in real time. *Computers and Industrial Engineering*, 58(2):249–256.

Ozturk, G., Bahadir, O., and Teymourifar, A. (2019). Extracting priority rules for dynamic multi-objective flexible job shop scheduling problems using gene expression programming. *International Journal of Production Research*, 57:3121–3137.

Park, S. C., Raman, N., and Shaw, M. J. (1997). Adaptive scheduling in dynamic flexible manufacturing systems: A dynamic rule selection approach. *IEEE Transactions on Robotics and Automation*, 13(4):486–502.

Planinic, L., Backovic, H., Durasevic, M., and Jakobovic, D. (2022). A comparative study of dispatching rule representations in evolutionary algorithms. *IEEE Access*.

Priore, P., Ponte, B., Puente, J., and Gómez, A. (2018). Learning-based scheduling of flexible manufacturing systems using ensemble methods. *Computers & Industrial Engineering*, 126:282–291.

Raghu, T. S. and Rajendran, C. (1993). An efficient dynamic dispatching rule for scheduling in a job shop. *International Journal of Production Economics*, 32(3):301–313.

Rajabinasab, A. and Mansour, S. (2011). Dynamic flexible job shop scheduling with alternative

process plans: an agent-based approach. *International Journal on Advanced Manufacturing Technology*, 54:1091–1107.

Sels, V., Gheysen, N., and Vanhoucke, M. (2012). A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *International Journal of Production Research*, 50:4255–4270.

Shady, S., Kaihara, T., Fujii, N., and Kokuryo, D. (2022). A novel feature selection for evolving compact dispatching rules using genetic programming for dynamic job shop scheduling. *International Journal of Production Research*, 60:4025–4048.

Shiue, Y. R., Guh, R., and Lee, K. (2012). Development of machine learning-based real time scheduling systems: using ensemble based on wrapper feature selection approach. *International Journal of Production Research*, 50(20):5887–5905.

Silva, Á. M. F. P. (2021). *Using Dimensional Aware Genetic Programming to find interpretable Dispatching Rules for the Job Shop Scheduling Problem.* Master's thesis, University of Porto.

Subramanian, S. (2024). Themes in machines: How is ai impacting manufacturing?

Tay, J. C. and Ho, N. B. (2007). Designing dispatching rules to minimize total tardiness. *Studies in Computational Intelligence*, 49:101–124.

Tay, J. C. and Ho, N. B. (2008). Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Computers and Industrial Engineering*, 54:453–473.

Teymourifar, A., Ozturk, G., Ozturk, Z. K., and Bahadir, O. (2020). Extracting new dispatching rules for multi-objective dynamic flexible job shop scheduling with limited buffer spaces. *Cognitive Computation*, pages 195–205.

van Hasselt, H., Guez, A., and Silver, D. (2015). Deep reinforcement learning with double q-learning, december 2015. *arXiv preprint arXiv:1509.06461*.

Wang, Y.-C. and Usher, J. M. (2005). Application of reinforcement learning for agent-based production scheduling. *Engineering Applications of Artificial Intelligence*, 18:73–82.

Wang, Y. F. (2020). Adaptive job shop scheduling strategy based on weighted q-learning algorithm. *Journal of Intelligent Manufacturing*, 31(2):417–432.

Wei, Y. and Zhao, M. (2004). Composite rules selection using reinforcement learning for dynamic job-shop scheduling. *2004 IEEE Conference on Robotics, Automation and Mechatronics*, pages 1083–1088.

Wu, S. Y. D. and Wysk, R. A. (1989). An application of discrete-event simulation to online control and scheduling in flexible manufacturing. *International Journal of Production Research*, 27(9):1603–1623.

Xie, J., Gao, L., Peng, K., Li, X., and Li, H. (2019). Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, 1(3):67–77.

Yang, H.-B. and Yan, H.-S. (2009). An adaptive approach to dynamic scheduling in knowledgeable manufacturing cell. *The International Journal of Advanced Manufacturing Technology*, 42(3-4):312–320.

Yin, Y., Stecke, K. E., and Li, D. (2017). The evolution of production systems from industry 2.0 through industry 4.0. *International Journal of Production Research*, 56(1-2):848–861.

Zhang, F., Mei, Y., Nguyen, S., and Zhang, M. (2021). Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job-shop scheduling. *IEEE Transactions on Cybernetics*, 51(4):1797–1811.

7. Appendices

Appendix A: Well-established DRs

Appendix B: Benchmark DR list

Appendix C: RTRS training effectiveness for different parameters

Table 12. Well-established DRs.

DR	Description
CR	Critical Ratio
EDD	Earliest Due Date
FIFO	First In First Out
LPT	Longest Processing Time
MDD	Modified Due Date
MOD	Modified Operation Due Date
ODD	Operation Due Date
SLACK	Job Slack
SOPT	Shortest Operation Processing Time
SIO	Shortest Imminent Operation processing time
SPT	Shortest Processing Time
RPT	Remaining Processing Time
WINQ	Work In The Next Queue
WIQ	Work In the Queue
NIQ	Number of jobs In Queue

For both QL and DQL, 45 parallel training runs were carried. These runs differ from each other as they rely on different state sets, learning rates, discount factors, and seeds.

After each training run, the most promising models are selected for further testing. Some runs revealed to be more successful than others, thus, *which parameters affect the training effectiveness the most?*

In QL, according to Figure 9, the state set is the dimension that most impacts the effectiveness of training. The learning rate and discount factor don't seem to have an appreciable impact on the number of promising models.

In DQL, by looking at Figure 10 the choice of learning rate and discount factor displays a stronger impact in the learning process than in QL. Moreover, the state set impact on DQL training is negligible, whereas it the most important dimension in QL. Finally, one of the seeds clearly under-performed when compared to the remaining four, which shows that DQL training is sensitive to the initial random policy.

Table 13. DR benchmark.

References	#DRs	Rule Source	Rule Expression
Silva (2021)	10	Tables 4.7, 4.8, and 4.9, only the best rules	CR^+ ; WINQ ; CR^++PT ; WINQ+PT ; CR^+*PT+WINQ ; CR^+*PT+WINQ+PT ; (S^++RPT)*(WINQ+PT)/RPT ; CR^+*PT+WINQ+2*RPT ; (S^++RPT)*(WINQ+PT)/(RPT+WINQ) ; WINQ + PT + ((S^+*PT)/(2*PT)
Anderson and Nyirenda (1990)	3	Equations 3 and 5	CR ; max(S/RPT*PT,PT) ; max(CR*PT,PT)
Braune et al. (2022)	1	Figure 5	JINQ+PT+2*min(JINQ, PT) + min(JINQ+PT, APT)
Ferreira et al. (2022)	2	R1 and R2 rules	2PT+PT·S^+/RPT+WINQ ; If (Slack > 0) Then PT+PT·CR^++WINQ Else 3*PT+WINQ - (WINQ/PT)
Holthaus and Rajendran (2000)	1	Equation 2	2*PT+WINQ+NPT
Tay and Ho (2007)	10	Rule_1 to Rule_10	TPT*(CT+AT+PT-3)+(CT*PT+AT*NOP)-(NOP*PT+2*PT+CT+1) ; (PT+CT+AT+2)*(RPT+PT+TPT) ; CT*TPT+5*NOP+3*AT ; DD*(AT+TPT+RPT+PT) ; (TPT+PT)*(CT+AT)+(DD-AT) ; 3*TPT+(PT/TPT+1)*(AT+RPT) ; 6*NOP+PT*CT*(PT+TPT) ; NOP+9*TPT+4*PT ; 4*TPT-2*NOP+3*DD+2*PT ; DD/TPT+2*TPT+PT+DD+AT
Tay and Ho (2008)	5	Rule_1 to Rule_5	AT+PT+2*TPT ; 7*TPT+11*PT+12*(NOP+AT) ; AT+2*PT+2*TPT+NOP ; (AT+DD)+2*(AT+TPT)+PT-NOP ; (AT+DD)+(TPT+PT)-2*(AT/NOP)
Hunt et al. (2016)	6	Figures 5, 6, 8, 9, and Expressions from Page 6	PT/RPT*(CT-DD)*((NROP+1)/NROP*AT) ; PT/RPT*(CT-DD)/DD ; if slack > 0 Then (-PT/(DD+OPW))*((CT*(DD-CT+RPT))/(DD+RPT)) Else (-PT/(DD+OPW)) ; PT*(min(CT,max(DD,CT)-RPT)-max(DD,CT))/RPT ; ATC-rule: (1/PT)*exp(-((DD-CT-PT-b*(RPT-PT))/(k*APT))$^+$) ; CoverT-rule: (1/PT)*(1-S^+/(k*b*TPT))$^+$
Ozturk et al. (2019)	4	Rules 8, 10, 11, and SOPT-rule	if (Slack > 0) Then Slack/NOP Else Slack*NOP ; CR'+PT, RPT+ODD ; TPT/(3+NOP-NROP) ; NROP*(EDD + NROP*((NROP+TPT)/(RPT-TPT)))
Planinic et al. (2022)	5	Table 14	3*PT+2*S^++(S^+)^2 ; PT*OPW-PT^2+S^+*DD+PT*S^+/(PT-S^+)+PT*(DD/PT+PT+DD)+S^+*(S^++PT)+DD ; S^+*PT+PT ; DD-PT+S^+ ; PT+2*DD+S^+
Raghu and Rajendran (1993)	1	Section 4	PT*exp(u)+PT*S/RPT*exp(-u)+WINQ
Sels et al. (2012)	21	Figure 1, only DRs for Mean Tardiness and for Large Job Shops	2*PT+RPT+FDD ; 2*PT+RPT+S ; PT+RPT+S ; 2*PT+RPT ; 2*PT+RPT+DD ; RPT+MOD ; RPT+PT ; RPT+PT+AVPRO ; PT+WINQ+RPT ; PT+OPW+RPT ; AVPRO ; RPT ; DD ; PT ; MOD ; PT+OPW ; PT+WINQ ; PT+OPW*FDD ; PT+PW+FDD ; PT+WINQ+NPT+WSL ; PT+S ; PT+WINQ+min(0,S)
Shady et al. (2022)	3	TWT, MT and MFT rules	max(NROP+max(NROP,PT),PT+WINQ)+max(max(S/NROP+PT,PT),NROP-S+WINQ) ; min((WINQ+S/NROP)/PT,max(CT,PT*NPT)) ; max(max(min(NPT,2*WINQ+PT),PT),WINQ-max(WINQ, PT))*PT
Teymourifar et al. (2020)	4	Rule 1 to Rule 4	PT/TPT ; NROP-(PT+AT)/NOP ; 2*TPT+NROP ; NROP-PT
Zhang et al. (2021)	2	Rules R1 and R2	PT+RPT-1/PT*(2*WIQ-1)-1/PT*(WIQ+RPT+(PT+RPT)/(WIQ-1+RPT)) ; NIQ*(PT-1)*(PT+RPT*max(PT,WIQ,max(WIQ,RPT)))+(NIQ+WIQ)*(1+max(RPT,NIQ-1))*max(WIQ,RPT)

Table 14. Node Description.

Node	Description
AT	arrival time
CT	current time
PT	operation processing time
NPT	next operation processing time
APT	average processing time of jobs in queue
RPT	job remaining processing time
TPT	job total processing time
CPT	amount of work already processed
AVPRO	average processing time of job's remaining operations
OPW	operation waiting time in queue
WINQ	processing time of the operations in the next queue
JINQ	number of operations in the next queue
NIQ	number of operations in queue
NOP	job total number of operations
NROP	job number of remaining operations
DD	job due date
ODD	operation due date
FDD	flow due date = AT+CPT
MOD	modified operational due date = max(AT+a*CPT,CT+PT)
MDD	modified due date = max(DD,CT+RPT)
S	job slack
S^+	job slack, if positive
WSL	waiting slack, if negative = min(S-WINQ,0)
CR	critical ratio = (DD-CT)/RPT
CR^+	critical ratio, if positive

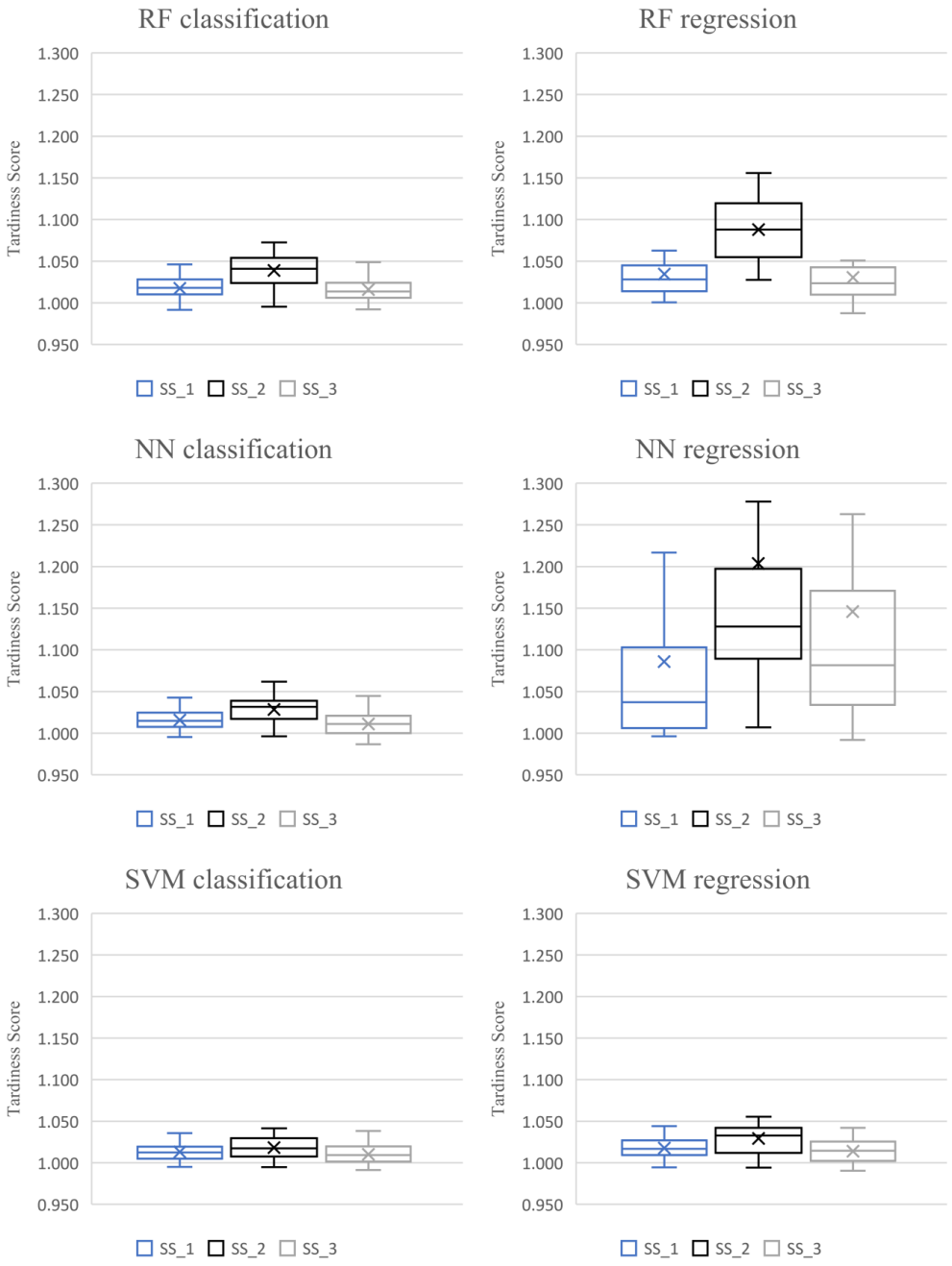


Figure 5. Validation instances performance of each PRS algorithm-task combination, grouped by state set. Models based in SS_2 present higher dispersion and worst results on average than SS_1 and SS_3 based models.

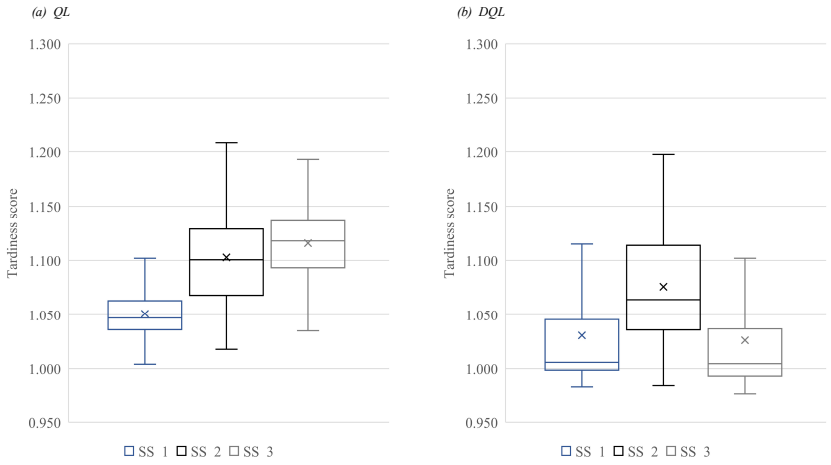


Figure 6. Performance of the RTRS models in the validation instances grouped by algorithm and by state set. For both QL and DQL, the SS_2 based models present worst performance on average and also higher variability than SS_1. Such was also observed for PRS. Regarding SS_3, DQL models present very good performance. In QL, SS_3 based models get worst on average.

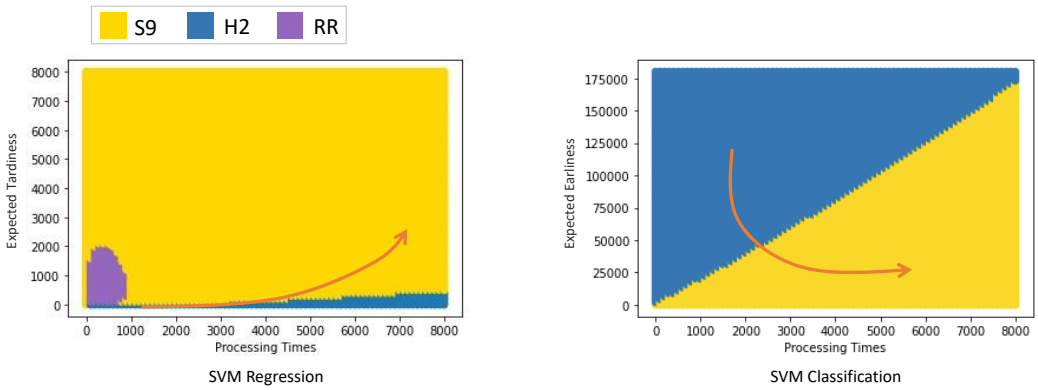


Figure 7. Models' DR suggestions for different utilisation and allowance values. When the job shop is more *tranquil*, i.e. tardiness is low and earliness is high, the H2 is preferred. Otherwise, when job shop load ramps up, S9 is more suitable.

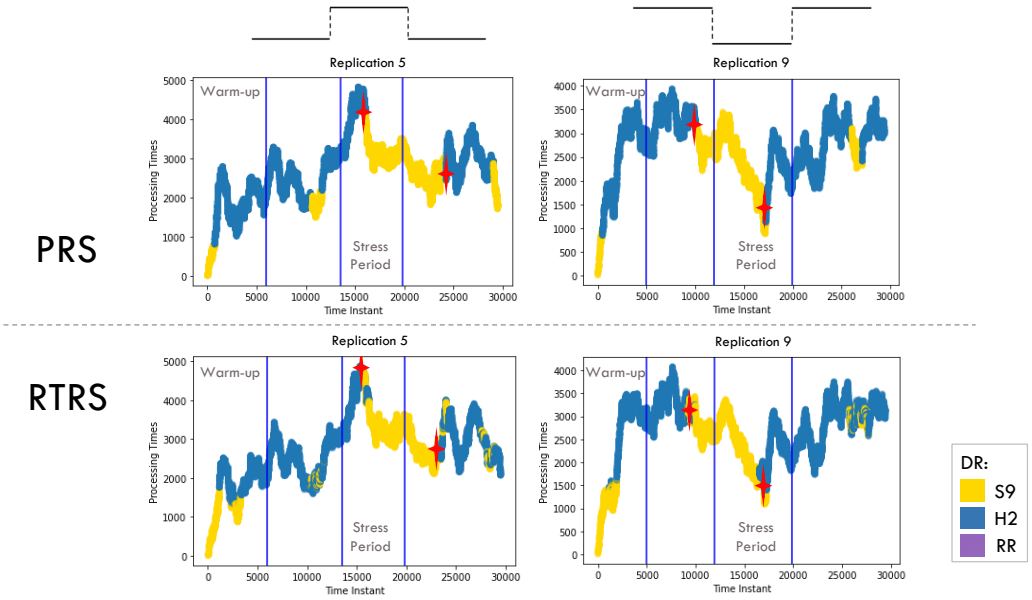


Figure 8. Selection of DR over time in non stationary instances

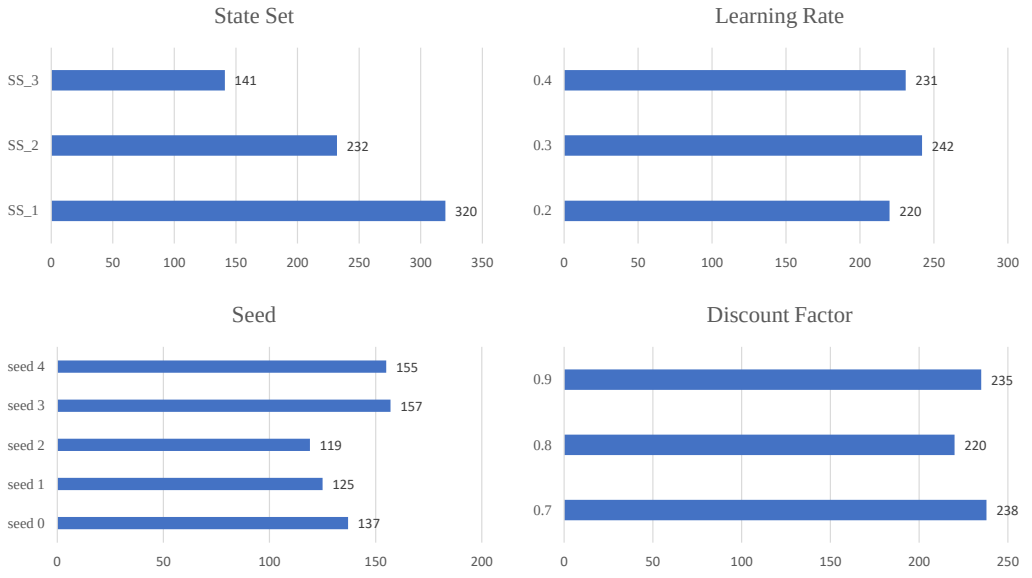


Figure 9. Impact of the training parameters (state set, learning rate, seed, discount factor) on the number of promising QL models. The state set was the dimension that most impact the training process.

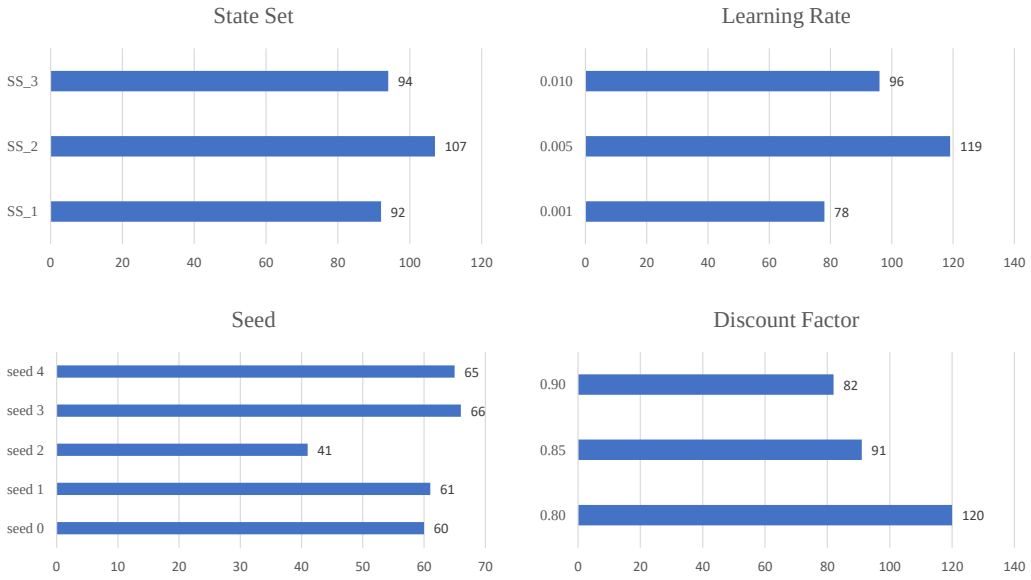


Figure 10. Impact of the training parameters (state set, learning rate, seed, discount factor) on the number of promising DQL models. The choice of learning rate and discount factor has a visible impact on the number of promising models found during training.